IDENTIFICATION OF ZERO-DAY EXPLOITS

A Project

Presented to the

Faculty of

California State Polytechnic University, Pomona

In Partial Fulfillment

Of the Requirements for the Degree

Master of Science

In

Computer Science

By

Jasmit Singh Mahajan

2023

SIGNATURE PAGE

PROJECT:	IDENTIFICATION OF ZERO-DAY EXPLOITS
AUTHOR:	Jasmit Singh Mahajan
DATE SUBMITTED:	Spring 2023
	Department of Computer Science

Dr. Ericsson Santana Marin
Project Committee Chair
Professor of Computer Science _____

Dr. Markus Eger
Professor of Computer Science _____

ACKNOWLEDGEMENTS

I would like to first thank Dr. Ericsson Santana Marin for agreeing to be my project mentor and being a part of my academic journey throughout my time at Cal Poly Pomona. Throughout my time, I thoroughly enjoyed being a part of your classes and your teachings helped gear me towards the Software Engineering Industry. I would also like to thank Dr. Markus Eger for agreeing to become my project committee member and for taking a high interest in learning more about my project. Your teachings and feedback will help me point out things that I would need to learn in the real-world and will help me grow in my career as a Software Engineer.

I would also love to thank my parents for pushing me and the sacrifices that they have made. My parents had taught me to always believe in myself and without their perseverance and guidance towards my academia and personal life, I would not be at the place that I am at today. They always allowed me to be the best version of myself and taught me how to be the person that I am today.

I wanted to also make a special shoutout to my sister, Simriti. My sister is a huge inspiration to me, and I could have not asked for a better sister ever. Even though she is just starting her college career, her maturity and humility are two important factors that I adapt towards academia and personal life and can’t thank her enough.

Lastly, I would like to thank my friends and the CPP Tamanna dance team to making my life at Cal Poly Pomona enjoyable and helping me take things one step at a time. I can’t wait to celebrate this achievement and my final moments of being a proud Bronco with you guys and for my future years to come.

ABSTRACT

Within the last couple of years, software vulnerability exploits have been an outstanding problem affecting different software systems and devices that are exposed to potential cyber-attacks. To mitigate these attacks, software vendors create patches to not allow these hackers to penetrate against these software vulnerabilities. However, these patches could occur before or after the exploitation date, meaning the date at which the software vulnerability was exploited in the wild. Different research studies have tried to evaluate the performance of these patched-up exploits using machine learning, but the results have not been substantial. There was also a body of work that described various methodological shortcomings that may occur with using machine learning. In this project, I will improve on previous research methods conducted to predict with high accuracy whether a software vulnerability is a zero-day or not. In addition, I would also address the reported methodological shortcomings with substantial results and provide an effective and detailed plan towards tackling patch prioritization. I will be testing and training various models with data gathered from different security repositories to achieve the desired results.

TABLE OF CONTENTS

SIGNATURE PAGE ii

ACKNOWLEDGEMENTS iii

ABSTRACT iv

LISTS OF FIGURES vii

LISTS OF TABLES viii

1. INTRODUCTION 1

1.1 Solution 2

2. BACKGROUND 4

3. RELATED WORK 6

4. DATASET 8

4.1 Important Features for Dataset 8

4.2 Data Cleaning 10

4.3 Training and Test Split 11

5. IMPLEMENTATION AND DESIGN 12

5.1 Machine Learning Pipeline 13

5.2 Baseline 14

5.3 Evaluation of Results 15

5.4 Results 16

6. CONCLUSION AND FUTURE WORK 17

7. REFERENCES 18

LISTS OF FIGURES

Figure 1: Overview of Software Vulnerability Lifecycle 5

Figure 2: Δt_patch_date Feature and Ground Truth 8

Figure 3: Has_edb and Has_zdi Features..... 9

Figure 4: Score of the Samples 9

Figure 5: Snippet of Results from Converter Files 10

Figure 6: Machine Learning Pipeline 13

Figure 7: Timeline Overview of the Baseline..... 14

Figure 8: SVM, Random Forest, and Adaboost Result 16

LISTS OF TABLES

Table 1: Feature List 12

INTRODUCTION

The National Institute of Standards and Technology (NIST) describes what a software vulnerability is: “a security flaw, glitch, or weakness is found within the software that is easily exploitable by an attacker”. [1] These vulnerabilities are discovered and publicly disclosed at an increasing rate. According to the National Vulnerability Database (NVD), in 2022 there were 26,448 software security flaws that were reported and in comparison to 2021, there was a 59 percent increase that was reported that are in critical condition. [2] But, from the number of vulnerabilities that are reported, only about 3% of these vulnerabilities are exploited by hackers giving organizations a smaller population for patching purposes. Vulnerability patching is checking and applying targeted changes to the operating systems, software, applications, and network components against malicious hackers that want to access and cause damage to users. [3]

Many of the patches that are created could occur either before or after the exploit date, meaning the date at which the software vulnerability was exploited in the wild. For the specific case when the patch is published only after the vulnerability gets exploited in the wild, we call this offensive a zero-day attack. This is because the users of the affected software component had zero-days to apply the patch, or in other words, they could not have the vulnerable component fixed by the time the exploitation happened. This is a slightly different definition of standard zero-day attacks, where malicious exploits target vulnerabilities that are completely unknown by the software vendors. As it would be very difficult to validate the quality of our model by using the standard zero-day definition, we made the proper adaptation without hurting the relevance of our predictions. In this

project, we only consider vulnerabilities that have an associated patch on NVD. All the other vulnerabilities for which there is not a patch have been discarded.

To figure out what software to patch up and what software vulnerabilities to prioritize, different organizations implemented the Common Vulnerability Scoring System (CVSS). This score provides a numerical indicator of the vulnerability severity, which is used by companies to measure risk. [4] However, research has been showing the CVSS score as a poor indicator of risk because of its inability to shortlist the vulnerabilities that would be exploited.

1.1 Solution

To solve this problem, different researchers had conducted experiments to use machine learning towards predicting software vulnerability exploitation through looking at patterns and various trends of data. But there are some methodological shortcomings that occur from this previous work: 1) class balancing is unrealistic, 2) separation between training and test data is conducted wrongly, 3) unrealistic and unrepresentative test data, and 4) meaningless benchmarks. This project addresses all these shortcomings and provides an effective method for patch prioritization. To train and test the proposed models, I created features from data collected from multiple security repositories such as CVE Lists, NVD, Zero Day Initiative, Exploitability Database, Darkweb Forum, and Symantec. The contributions that the project has created were the following:

- Resolved the methodological shortcomings alongside determining zero-day attacks from software vulnerabilities.
- Evaluating various performances of different machine learning algorithms, with a performance enhancement between Random Forest (0.258) and SVM (0.545).

- Achieving a 200% performance increase in comparison to the previous work that was conducted.

BACKGROUND

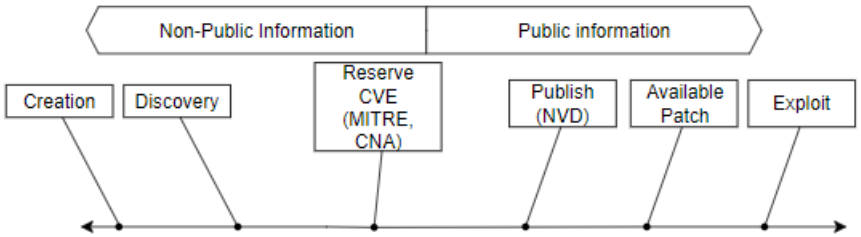
To achieve the goal of this project, it is important to understand the software vulnerability lifecycle and its variations that could happen.

CVE Reserving. Once it gets reported, the CNA (CVEs Numbering Authorities) or MITRE would assign a CVE number if the following constraints were satisfied: 1) The vulnerability exploit causes violations against security policies and 2) there has not been a CVE that has been assigned to the issue. Afterwards, once this is approved, the software vendor is alerted that this vulnerability flaw could indicate a cyber threat and allows for a specific period to have patches before the vulnerability is publicly disclosed. For this research, only a security issue with an assigned CVE identifier is considered a vulnerability and only those with a patch are used for training the model.

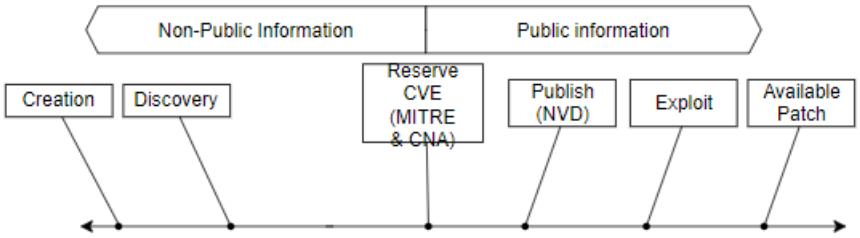
Vulnerability Disclosure. Once a vulnerability is checked and the period given to the vendor is over, the NVD, which is a comprehensive vulnerability, that publicly discloses different types of information about the vulnerability, including the affected software components, severity score, linked to proof of concept, and other various metrics.

Vulnerability Exploitation. This is the stage at which a vulnerability would get exploited in the wild. The way we would get this information is through the logs of the Symantec security firm. This stage would also help other organizations to assess the likelihood of an exploit in comparison to previous severity scoring systems.

You can see an overview of the Software Vulnerability lifecycle in Figure 1 where the a) Non-Zero Day Attack happens when the exploit happens after the patch and b) a Zero-Day Attack happens when the exploit happens before the patch.



a) Non Zero-Day Attack Scenario



b) Zero Day Attack Scenario

Figure 1: Overview of Software Vulnerability Lifecycle

RELATED WORK

The related work to this project begins by having a foundation of Vulnerability Exploitation Prediction and Social Network Analysis. In terms of Vulnerability Exploitation Prediction, Almukaynizi et. Al [5] explains how applying unimportant patches is undesirable since this might disrupt business processes while taking systems down, for instance. [5] In terms of Social Network Analysis, there were a few works that described how to identify malicious actions in online social networks. For instance, an application called SybilRank relied on social network measures to identify fake accounts in large scale social media platforms. [6]

Transitioning on to more of Almukaynizi's work, it also emphasized a model that was based on six sets of features and is used within this project as well. The first set of features that was presented is using the most frequent words within the dataset. In order to do so, the study used TF-IDF to vectorize text from NVD and Darkweb posts. The second set of features that the study discusses is the CVSS base score and its vectors. This score is assigned by NVD to each CVE and informs the calculated severity of each vulnerability. There are six different criteria that are important towards determining the CVSS score: Access Vector, Access Complexity, Authentication, Confidentiality, Integrity, and Availability. In this project, I have included all these important criteria in comparison to previous work to develop the features of the dataset. The third set of features include the DW language which is the categorical data that uses the language of the website. The final three sets of features are binary and encompass if a proof of concept (PoC) is available within EDB. The EDB is a collection of PoC exploits that are maintained by the Offensive Security Training and it is important to inform whether a

PoC exploit is available for a particular CVE. [7] They check for ZDI and the Dark Web. ZDI maintains a database of vulnerabilities submitted by security researchers to develop patches for the reported vulnerability before it is disclosed to the public. [7] This project will hone the patches that occur before the exploitation date to indicate a Zero Day Attack. With these features, Almukaynizi's work [5] was able to retrieve a F1 score of 0.4 and will serve as a baseline to be overcome.

As discussed before, there are some methodological shortcomings that are listed by the work conducted in IWSPA [5]. They mainly included machine learning work that tried to predict exploitation. The first shortcoming is the class imbalance, which shows that the balance between the positive and negative classes were artificially created on the test set. The second shortcoming is the separation of training and testing data leading to time intermixing behavior from using future events to predict the past behavior. The third shortcoming is lack of meaningful benchmarks by just using the CVSS score as the main benchmark. The last shortcoming that was reported was the unrealistic test data used, including information not available at the time of prediction.

DATASET

The data that is gathered for the development of the dataset is from the following databases: NVD, MITRE, EDB, ZDI, and a set of Darkweb hacker data collected by Cyr3con. For the Darkweb data, it was retrieved with the use of a crawler service, Cyr3con, that had filtered out specific features that weren't related to cybersecurity.

4.1 Important Features for Dataset

In this work, we use data from 2015, 2016, and 2017. The class labels were represented with a Boolean value for the exploitation of the software vulnerability or not in regard to the Δt_patch_date, which is the difference between the exploitation_date and the disclosed_patch_date, which was retrieved from the NVD, of the CVE shown below in Figure 2.

delta_t_patch_date integer	train_label boolean
-1	true
8	false
4	false
-15	true
6	false
55	false

Figure 2: Δt_patch_date Feature and Ground Truth

For the EDB and ZDI data that was used, the has_edb and has_zdi was created as two distinct features within the dataset to represent whether the CVE exists within the Exploit Database or the Zero-Day Initiative Database. The value of these two features is represented by a Boolean value and are shown in Figure 3 below.

has_edb boolean	has_zdi boolean
false	false
true	true
true	false
true	false
true	false
false	true

Figure 3: Has_edb and Has_zdi Features

In order to analyze the quality of our collected data and avoid the mentioned methodology shortcomings, we calculate a score for each sample by following this strategy: we assign 1 if the $\Delta t_publish_date$, difference between the exploit_date and the nvd_publish_date, is greater than 0, 2 if the $\Delta t_publish_date$ is 0, 3 if the $\Delta t_publish_date$ is less than 0. Then, we only use samples with a score of 1 and 2, as illustrated in Figure 4.

score integer
2
1
1
2
1
1

Figure 4: Score of the samples

4.2 Data Cleaning

For the cleaning and filtering of the data, the data which does not have a disclosed_patch_date associated is ignored from our analysis.

In terms of ground truth, only the CVE's that had spanned from 2015 to 2017 were used. In order to retrieve the data from 2015 to 2017, converter files were used per year that would retrieve the necessary features and would put this into a generated csv file. Here is a snippet of the results of these converter files shown in Figure 5 below.

```
,cve_id,cwe_id,cve_publish_date,cve_description,cvss2_accessComplexity,cvss2_accessVector,cvss2_authentication,cvss
0,CVE-2017-0001,NVD-CWE-noinfo,2017-03-17T00:59Z,"The Graphics Device Interface (GDI) in Microsoft Windows Vista SP
1,CVE-2017-0002,NVD-CWE-noinfo,2017-01-10T21:59Z,"Microsoft Edge allows remote attackers to bypass the Same Origin
2,CVE-2017-0003,CWE-119,2017-01-10T21:59Z,"Microsoft Word 2016 and SharePoint Enterprise Server 2016 allow remote a
3,CVE-2017-0004,CWE-20,2017-01-10T21:59Z,"The Local Security Authority Subsystem Service (LSASS) in Microsoft Windo
4,CVE-2017-0005,CWE-119,2017-03-17T00:59Z,"The Graphics Device Interface (GDI) in Microsoft Windows Vista SP2; Windo
5,CVE-2017-0006,CWE-119,2017-03-17T00:59Z,"Microsoft Excel 2007 SP3, Office Compatibility Pack SP3, Excel Viewer, a
6,CVE-2017-0007,CWE-20,2017-03-17T00:59Z,"Device Guard in Microsoft Windows 10 Gold, 1511, 1607, and Windows Server
7,CVE-2017-0008,CWE-200,2017-03-17T00:59Z,"Microsoft Internet Explorer 9 through 11 allow remote attackers to obtai
```

Figure 5: Snippet of Results from Converter files

4.3 Training and Test Split

The split that is used in this project is a 70-30 split between training and testing based on the assigned date of the MITRE database. The number of samples that form the training data was 145, of which 30 are positive and 115 are negative. For the test samples, the number of samples included was 63 and there were 5 positive samples and 58 negative samples. The reason for which the 70-30 split was used is because in comparison to the 80-20 split, there were more instances of positive results that were shown within the test set.

IMPLEMENTATION AND DESIGN

There are five different sources that this project will examine to engineer features: Darkweb, NVD, MITRE, EDB, and ZDI. The ground truth will be from the Symantec database with various samples of vulnerability exploitation. In terms of the data types, there are two data types that each of the sources could have: Numeric and Categorical. These two data types will help choose the correct method to extract the features. To see the details of the features, see Table 1 below.

Table 1: Feature List

Source	Data Type	Features Extracted
Darkweb	Numeric	Distinct Post Distinct User
NVD	Categorical	CVSS Base Vectors
	Numeric	Reference Listed Affected Products and Vendors
MITRE	Numeric	Time Difference between MITRE and NVD
EDB	Numeric	EDB Exists
ZDI	Numeric	ZDI Exists

5.1 Machine Learning Pipeline

Throughout the ML process, we first preprocess the data in order to feed training and testing samples to a machine learning model. To view the pipeline that will be used, please refer to Figure 5 shown below. As our ground truth to the project, we use the Symantec database and create our models to predict with high accuracy whether a given zero-day or not according to the zero-day definition provided before.

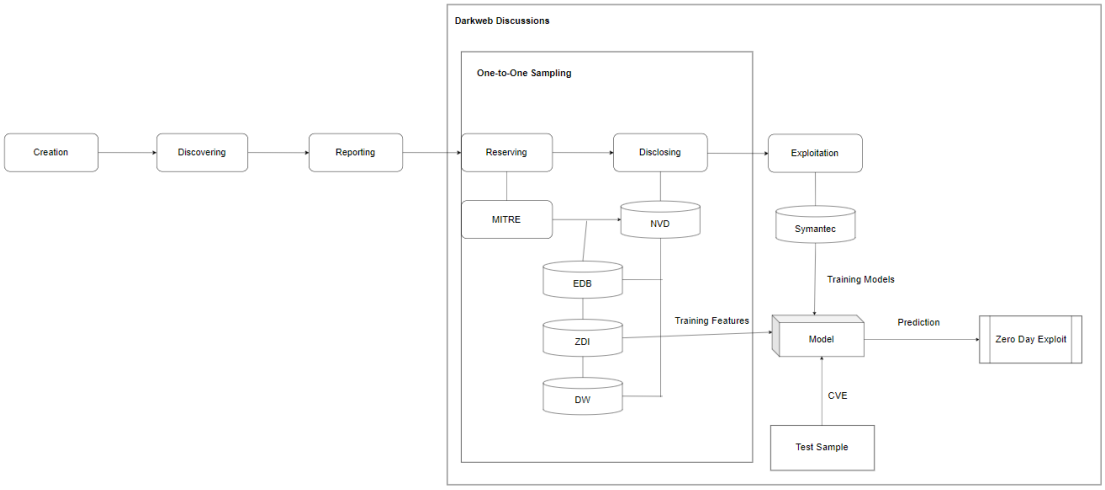


Figure 6: Machine Learning Pipeline

5.2 Baseline

Finally, we explain below how the CVSS score is used as a baseline. First, we are showing below how the CVSS score does not perform as expected. If the CVSS score is greater than a given threshold, the model would predict true otherwise it will be false. The range of which a CVSS score is determined is from 0.0 to 10.0. To see the behavior of the baseline, please see Figure 6 below. Currently, the highest F1 score is depicted at 0.12 and this is just using the CVSS score, which behaves poorly. Thus, in order to have a more meaningful baseline for this project, we will use Mohammad's paper discussed in the Introduction. [7]

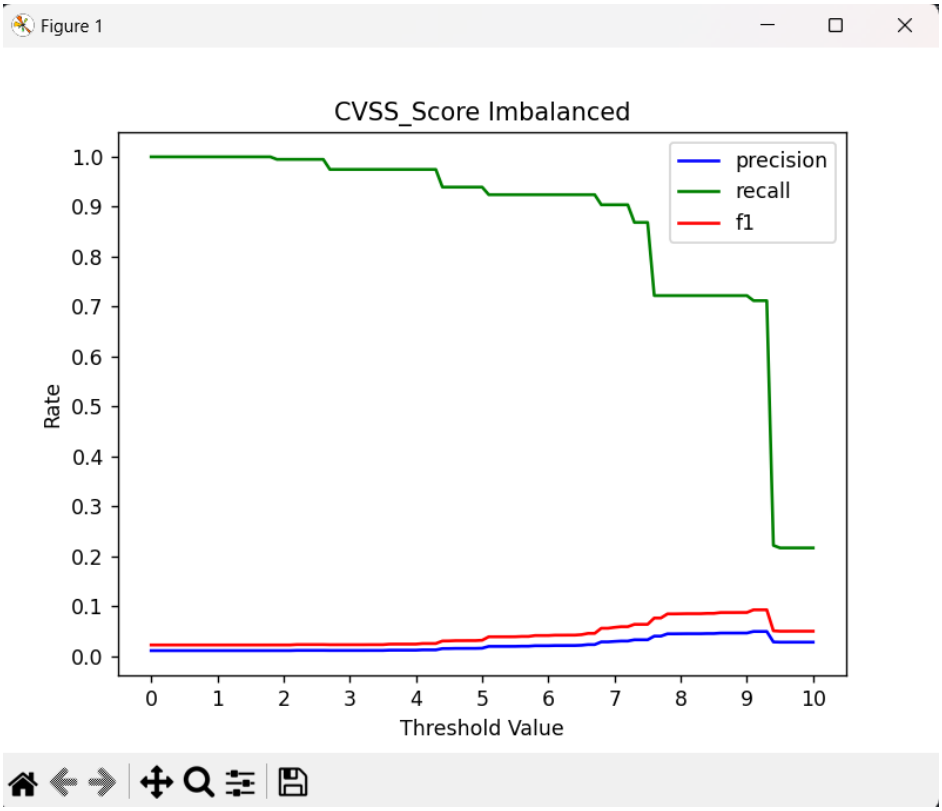


Figure 7: Timeline Overview of the Baseline

5.3 Evaluation of Results

The results of the project are evaluated with four different metrics: Accuracy, Precision, Recall and F1 score. Since our problem is strongly imbalanced towards non-zero-day vulnerabilities, the accuracy of the model will not be significant in our analysis of the project. A better metric in comparison to the Accuracy is F1 because it uses the combination of precision and recall. High F1 correlates to high precision and recall. Below are the formulas to calculate each of these metrics:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

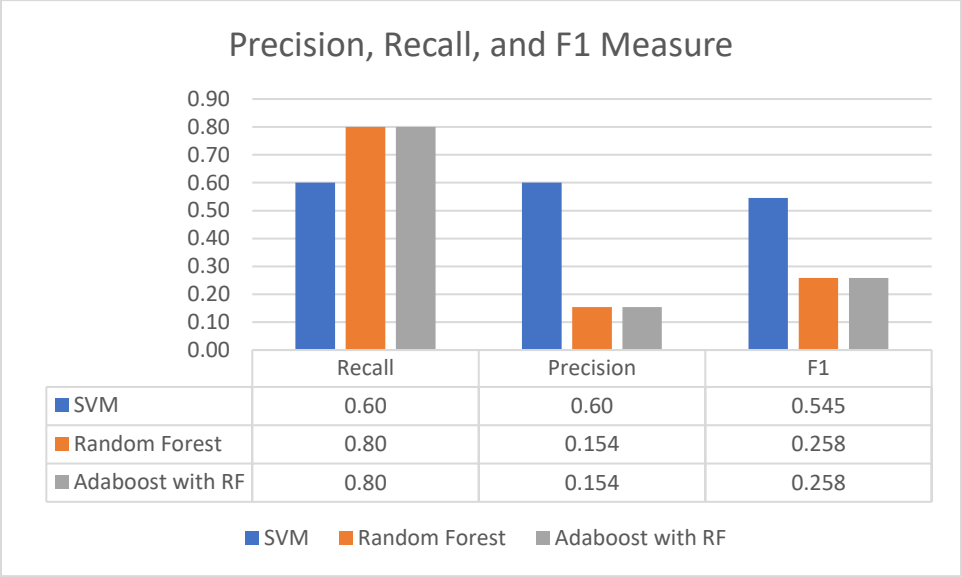
$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} = \frac{2 * TP}{2 * TP + FP + FN}$$

5.4 Results

For the Support Vector Machine (SVM) Model, the highest F1 score was retrieved at 0.545, which is a 200% increase in comparison to previously created results.

There were also other models that were tested, notably Adaboost and Random Forest, that while having a high recall rate, the precision of the model is relatively low. Figure 7 below below are the results that were retrieved from SVM, Random Forest and Adaboost respectively.



	Recall	Precision	F1
SVM	0.60	0.60	0.545
Random Forest	0.80	0.154	0.258
Adaboost with RF	0.80	0.154	0.258

Figure 8: SVM, Random Forest, and Adaboost Result

To generate the models, all information related to the exploitation_date was excluded to avoid cheating.

CONCLUSION AND FUTURE WORK

Throughout this project, it addresses a binary classification problem to predict whether a given software vulnerability is a zero-day or not according to our zero-day definition. This project demonstrates involvement and modification of new features leading to the increase of the result and performance of the model by over 200 percent. This project also demonstrates the four shortcomings that were to be tackled: class imbalance, wrongly classifying test and training data, lack of realistic test data, and extraneous benchmarks that are meaningless. In the future, some improvements that could be implemented on the project could be the use of more data from years after 2017 and using TF-IDF to vectorize NVD and Darkweb information.

REFERENCES

[1] Editor, CSRC Content. "Software Vulnerability - Glossary: CSRC." *CSRC Content Editor*, https://csrc.nist.gov/glossary/term/software_vulnerability.

[2] Targett, Ed. "Analysis of 26,000+ Cves in 2022 Shows Shocking Rise in..." *The Stack*, 11 Jan. 2023, https://thestack.technology/analysis-of-cves-in-2022-software-vulnerabilities-cwes-most-dangerous/#:~:text=A%20record%2026%2C448%20software%20security,and%20Exposures%20(CVEs)%20data..

[3] Contributor, Staff. "Vulnerability and Patch Management Guide." *DNSstuff*, 12 Oct. 2020, https://www.dnsstuff.com/vulnerability-and-patch-management.

[4] "Common Vulnerability Scoring System SIG." *FIRST*, https://www.first.org/cvss/.

[5] Mohammed Almukaynizi et al. "Predicting Cyber Threats through the Dynamics of User Connectivity in Darkweb and Deepweb Forums". In: 2017 (CSS)

[6] Qiang Cao, Michael Sirivianos, Xiaowei Yang, and Tiago Pregueiro. 2012. Aiding the detection of fake accounts in large scale social online services. In Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation. USENIX Association, 15–15.

[7] Mohammed Almukaynizi et al. "Proactive identification of exploits in the wild through vulnerability mentions online". In: 2017 International Conference on Cyber Conflict (CyCon U.S.) 2017, pp. 82–88. DOI: 10.1109/CYCONUS.2017.8167501.